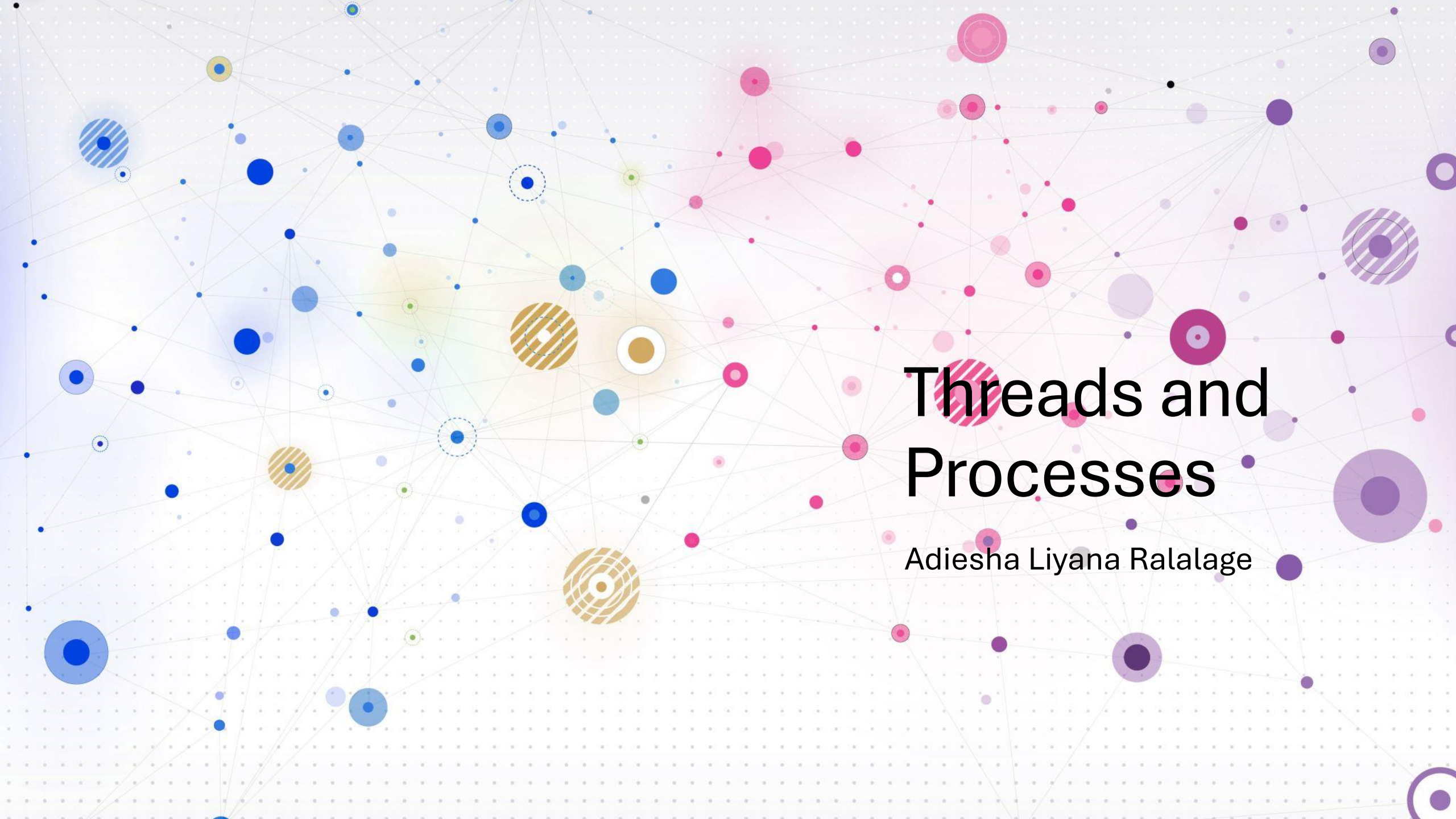


Threads and Processes

Adiesha Liyana Ralalage

Process characteristics

Resource ownership

- A process has a virtual address space containing its code, data, and execution stacks.
- The OS protects process resources from unauthorized access by other processes.

Scheduling/Execution

- Execution may be interleaved with other processes.
- In the single-threaded model, each process has an execution state (Ready, Running, or Blocked) and a scheduling priority.
- The OS selects a ready process and dispatches it to a CPU.

- Resource ownership and execution are distinct responsibilities that can be managed separately.
- Modern operating systems treat them independently.
- Conceptual Separation: Resource ownership is about what a process possesses (e.g., memory, files), while scheduling/execution is about what a process does (e.g., running its code).
- A process can own resources without executing (e.g., a blocked process holding a file lock) or execute without owning significant resources (e.g., a lightweight computation).

Implications

- This independence allows the OS to manage resources and execution separately. For example:
- Resource Management: The OS can allocate/deallocate resources (e.g., memory or I/O devices) independently of whether the process is currently running.
- Scheduling: The OS can prioritize, and dispatch processes based on their execution state and priority, regardless of their resource holdings.

Processes and Threads

- The unit of dispatching is referred to as a **thread** or **lightweight process.**
- The unit of resource ownership is referred to as a **process** or **task.**
- **Multithreading** - The ability of an OS to support multiple, concurrent paths of execution within a single process.

Multithreading approaches

- A single thread of execution per process, in which the concept of a thread is not recognized, is referred to as a single-threaded approach

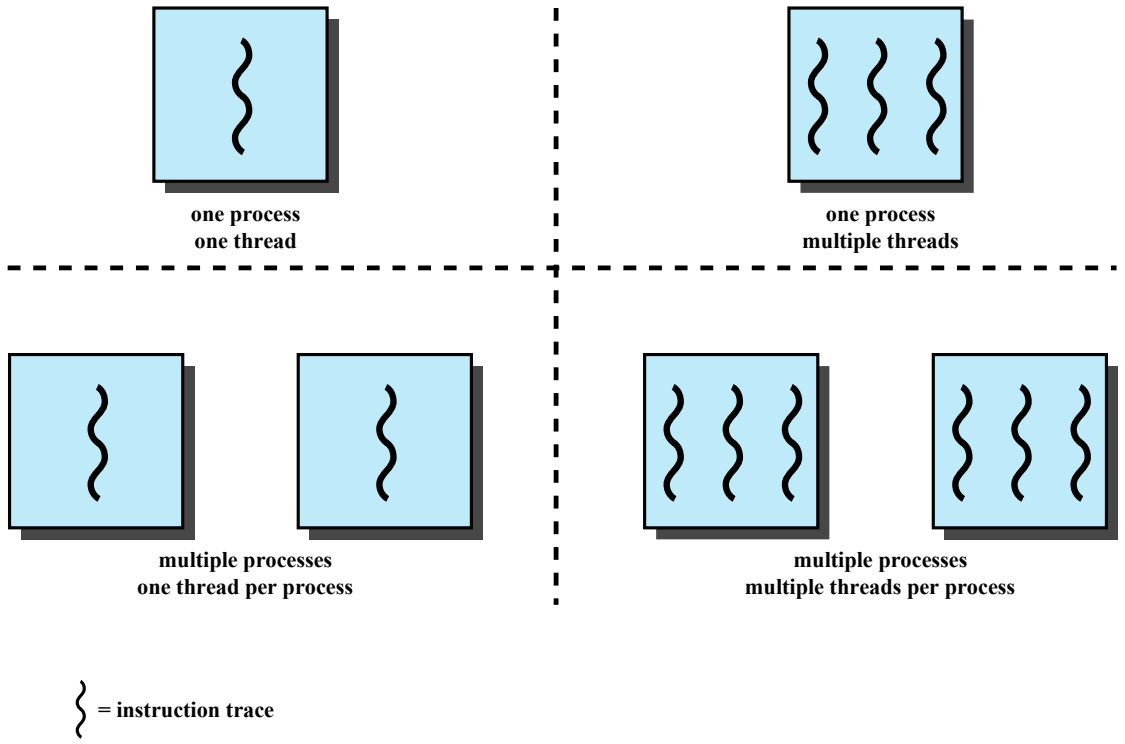


Figure 4.1 Threads and Processes

Multithreaded approaches

- The right half of Figure 4.1 depicts multithreaded approaches
- A Java run-time environment is an example of a system of one process with multiple threads

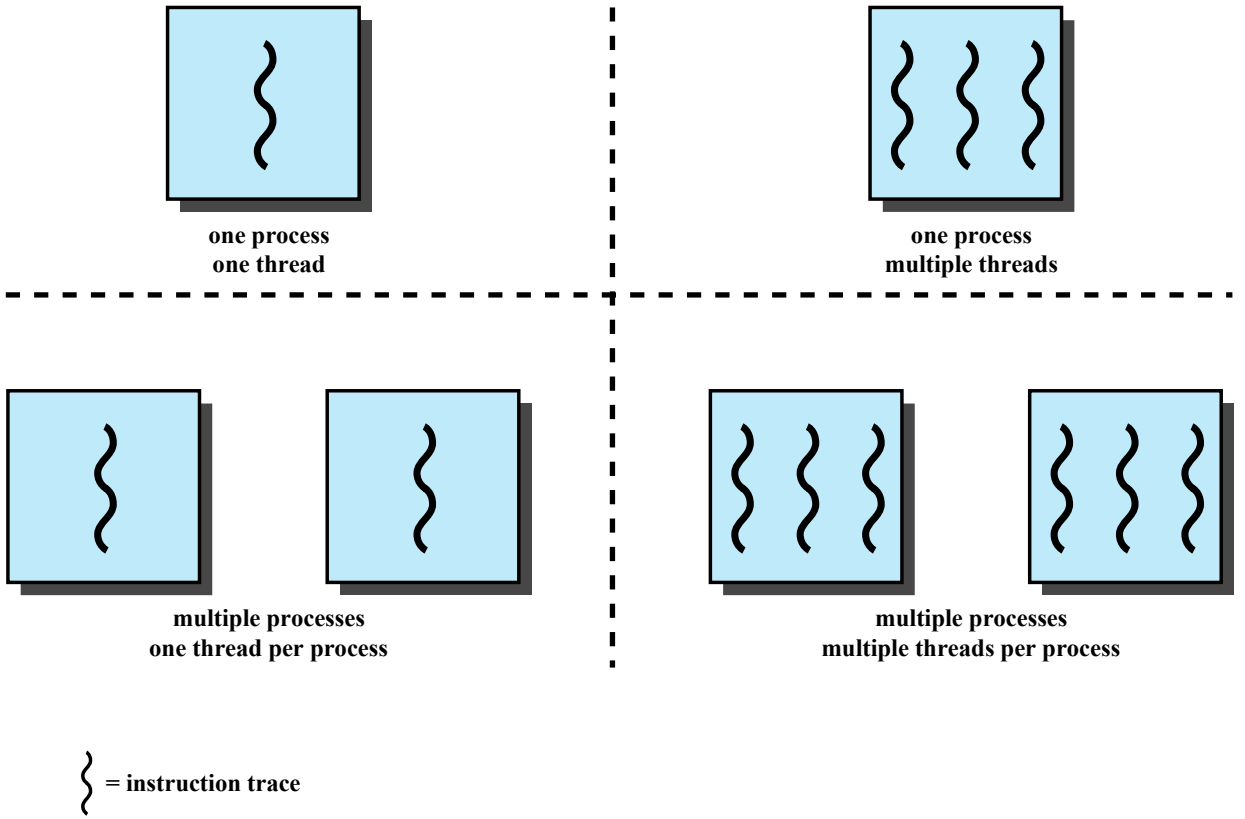


Figure 4.1 Threads and Processes

Process

- Defined in a multithreaded environment as “**the unit of resource allocation and a unit of protection**”
- Associated with processes:
  - A virtual address space that holds the process image
  - Protected access to:
    - Processors
    - Other processes (for interprocess communication)
    - Files
    - I/O resources (devices and channels)
- Within a process, there may be one or more threads.

One or More Threads in a Process

Each thread has:

- An execution state (Running, Ready, etc.)
- A saved thread context when not running
- An execution stack
- Some per-thread static storage for local variables
- Access to the memory and resources of its processes, shared with all other threads in that process

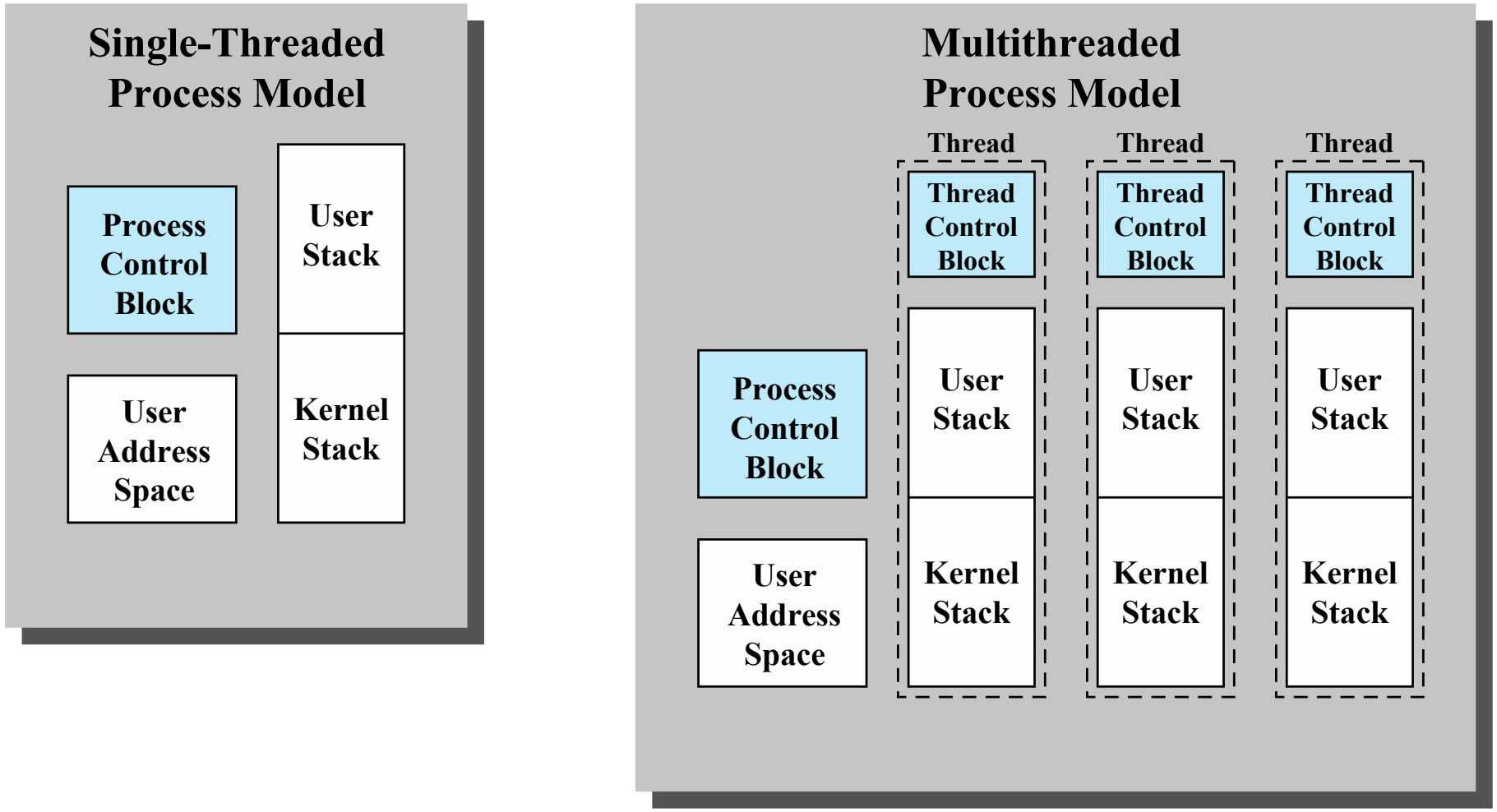
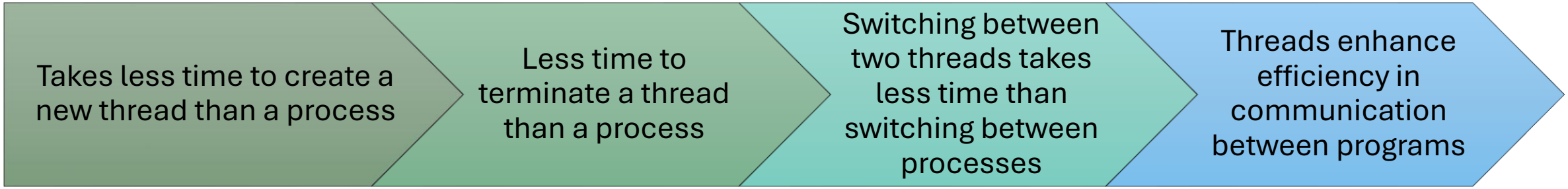


Figure 4.2 Single Threaded and Multithreaded Process Models

Key benefits of Threads



Thread Use cases in a Single-User System

- Foreground and background work.
- Asynchronous processing.
- Speeding up of execution.
- Modular program structure.

Threads

- In an OS that supports threads, scheduling and dispatching is done on a thread basis.
- Most of the state information dealing with execution is maintained in thread-level data structures.
  - Suspending a process involves suspending all threads of the process.
  - Termination of a process terminates all threads within the process.

Thread Execution States

- The key states for a thread are:
  - Running
  - Ready
  - Blocked
- Normally, we don’t have a suspend state as it is a concept associated with process rather than threads.

- Thread operations associated with a change in thread state are:
  - Spawn
  - Block
  - Unblock
  - Finish